Same Direction, Different Noise

The full story of the project, told from zero:
the problem, the literature, the gap, every experiment and why we ran it,
and where we stand

Mohamed Ayman
October 18, 2026

Abstract

This document assumes no background. It explains how language models are trained with reinforcement learning to solve math problems, what the research literature says and where it has gaps, and then tells, in order, everything this project has done: an earlier idea (STOP) and why it failed, a theory of what each generated answer is worth (WORTH), the measurements and experiments that tested it, the change of direction they forced, the main finding (rewarding right answers and penalizing wrong answers push the model the same way on average, and differ only in noise), an independent review that scored the work 4 out of 10 for ICML and what we changed because of it, and the two experiments now running that decide what kind of paper this becomes. Every experiment is listed with its question, its reason, its design, its pre-registered prediction, and its result (Appendix A).

Contents

1 The problem in plain words **3**

2 Background you need **3**

3 The literature, and where the gaps are **4**

4 The earlier idea (STOP) and why it failed **6**

5 What is each generated answer worth? (WORTH) **7**

6 Measuring the four numbers **8**

7 First tests of the theory **9**

8 The literature check and the change of direction **11**

9 PSR against NSR (Pre-registration 4) **12**

10 Matched accuracy, and the NSR paper's own implementation **13**

11 Same average, graded noise (Pre-registration 7) **15**

12 Changing the objective, and grading the reasoning (Pre-registration 8) **15**

13 What the Qwen experiments say together **16**

14 An independent review, and what we changed **16**

15 The two experiments running now **17**

16 Where we stand, and the path to a paper **19**

A Experiment tracker **20**

B Glossary **22**

C How we work, and engineering problems fixed **22**

1 The problem in plain words

Large language models can now solve competition math problems. A big part of how they got there is a training method called **reinforcement learning with verifiable rewards (RLVR)**: the model writes an answer to a math question, a simple program checks whether the final number is right, and the model is nudged to make answers like the right ones more likely and answers like the wrong ones less likely. Repeat this over thousands of questions and the model improves. This recipe is behind DeepSeek-R1 and many open models.

The loop has many design choices that look small but are not:

- **How many answers to write per question.** Writing answers is most of the cost of training.
- **Which answers to push up or down, and by how much.** For example, reward only the right answers, or only penalize the wrong ones, or both.
- **What to aim for.** Getting one answer right ("pass@1") or getting at least one right out of several tries ("pass@k", a measure of diversity).

Every one of these choices changes three things at once: *which direction* the model is pushed, *how noisy* that push is, and *how much it costs*. Papers usually compare such choices by trial and error, often with unequal compute, one random seed, and small test sets. This project set out to make the effects exact and measurable, and then to test them carefully.

The headline result so far: **rewarding right answers and penalizing wrong answers push the model in the same direction on average; what separates them is noise, and wrong answers carry most of it.** A well-known paper (Zhu et al., NeurIPS 2025) argued that penalizing wrong answers is special because it preserves diversity. In our experiments it behaves like a noisier, slower version of rewarding right answers. Whether that holds for another model family, and whether the noise is truly the *cause*, are the two experiments running now.

2 Background you need

The model as a "policy"

A language model given a question x defines a probability $\pi(y \mid x)$ for every possible answer y (a sequence of tokens). Sampling from it at temperature 1 gives a random answer. In RL language the model is a *policy*: it chooses what to say.

Reward. A verifier compares the answer's final boxed number with the known solution: reward $r = 1$ if right, $r = 0$ if wrong. For a question, the model's **success probability** p is the chance that a sampled answer is right.

pass@1 and pass@k. pass@1 is the average success probability over test questions (accuracy). pass@k is the fraction of questions where at least one of k sampled answers is right. With n samples of which c are right, the unbiased estimate is $1 - \binom{n-c}{k} / \binom{n}{k}$. pass@$k$ at large k measures *coverage* or *diversity*: whether the model can still find the right answer somewhere among many tries. Training usually raises pass@1 and can lower pass@k at large k, because the model becomes confident and repetitive.

Policy gradient. To increase the expected reward, move the model's parameters θ along its gradient. The classic identity (REINFORCE, Williams 1992) is

$$\nabla_\theta \, \mathbb{E}[r] = \mathbb{E}\big[r \, \nabla_\theta \log \pi(y \mid x)\big].$$

The vector $\ell(y) = \nabla \log \pi(y)$ is the answer's **score**: the direction that makes that answer more likely. So the training update is a sum over sampled answers, $g = \sum_i A_i \, \ell(y_i)$, where A_i is the answer's **advantage**: positive pushes the answer up, negative pushes it down.

Baselines. Because $\mathbb{E}[\ell(y)] = 0$ (probabilities sum to one), subtracting a constant b from every reward, $A = r - b$, does not change the *average* update. It does change its *noise*. This is classical (Williams 1992; Greensmith, Bartlett and Baxter 2004) and matters a lot later.

GRPO and Dr. GRPO. The standard recipe samples a *group* of n answers per question (here $n = 8$) and uses the group's mean reward as the baseline. GRPO (DeepSeekMath, 2024) also divides by the group's standard deviation and averages each answer's loss over its tokens; Dr. GRPO ("GRPO done right", Liu et al. 2025) showed those two divisions bias training and removed them, giving $A_i = r_i - \bar{r}$. Example: if 2 of 8 answers are right, right answers get $+0.75$ and wrong ones -0.25. Dr. GRPO is our main baseline.

PSR and NSR. Two extreme choices studied by Zhu et al. (2025): *positive sample reinforcement* (PSR) gives $A = 1$ to right answers and 0 to wrong ones (only reward successes); *negative sample reinforcement* (NSR) gives $A = -1$ to wrong answers and 0 to right ones (only penalize failures). W-REINFORCE, their recommendation, gives 0.1 to right and -1 to wrong.

Signal and noise. Each update is a random vector: its average (over the random answers) is the *signal*, the direction you want; its fluctuation around that average is the *noise*. A noisy update wastes part of each step. In ordinary deep learning, the *gradient noise scale* (McCandlish et al. 2018) says how many samples per step are needed before noise stops mattering.

Adam. The optimizer used everywhere. It keeps running averages of each coordinate's gradient ("momentum") and squared gradient, and divides the first by the square root of the second. A consequence that matters here: more noise in the gradient inflates the denominator, so Adam takes *smaller effective steps* along the signal. Noise therefore slows Adam down.

On-policy. We always sample fresh answers from the current model and update once on them. Many systems (for example the verl framework) reuse each batch of answers for several updates, with a "clipped" loss that limits how far the model moves from the one that wrote them.

Pre-registration. Before each experiment we wrote down, in a dated file, the design, the prediction and the rule that decides the verdict, so that results could not quietly reshape the question. All files are in `experiments/training/PREREGISTRATION_*.md`.

3 The literature, and where the gaps are

3.1 RLVR recipes

GRPO (Shao et al., DeepSeekMath, 2024) made group-based RL standard for math; DeepSeek-R1 scaled it. Dr. GRPO (Liu et al. 2025) removed GRPO's length and standard-deviation biases. DAPO (Yu et al. 2025) added "dynamic sampling": drop questions whose 8 answers are all right or all wrong (they carry no learning signal under a group-mean baseline) and refill the batch.

3.2 Spending compute on answers: adaptive rollout collection

Since writing answers dominates cost, many 2025–2026 methods decide adaptively how many answers each question gets: probe a few answers and continue only on questions with mixed results (SPEED-RL, SARA, Pilot-Commit, GRESO), allocate a budget across questions (Knapsack RL, Reinforce-Ada, PAIR), or sample until a correct answer appears. Our review of these papers found recurring evaluation flaws:

- gains plotted against training *steps* while spending 2 to 3 times more answers per step (for example, SARA's +6.8 points comes with 3.3 times the answers);
- learning rates not retuned, single seeds, 30-problem benchmarks, best checkpoints chosen on the test set;
- the claimed mechanism ("lower variance") asserted but never measured.

3.3 Theory of the update

Davis and Recht (2025) and ARCUS (2026) derive the *average* update of fixed-size groups: on average, a group's update points along ∇p, scaled by a function of p. McCandlish et al. (2018) introduced the gradient noise scale; Bansal, Mohri, Abreu, Alvarez-Melis and Kakade ("Critical Batch Size for LLM Policy Optimization", HiLD workshop at ICML 2026) apply it to GRPO, split the noise into a between-question and a within-question part, and find the within-question part dominant. Li et al. (2026) study square-root learning-rate scaling; IsoCompute (2026) studies answers per question; Malladi et al. (2022) explain square-root scaling for Adam.

3.4 Assumptions about noise

Recent variance-reduction papers model the noise with simplifying assumptions. VSR (Wang, Liu and Wang, October 2026) states as Assumption B.3 that the expected squared score norm does not depend on whether the answer was right; BICC (2026) uses a similar approximation. Nobody, to our knowledge, had measured whether right and wrong answers have different gradient sizes.

3.5 Does RL add reasoning, or only sharpen it?

Yue et al. (2025) showed that RL-trained models beat their base models at pass@1 but can lose at large k: RL may sharpen what the base model already does rather than add new abilities. Wen et al. (2025) introduced **CoT-Pass@k**, which counts an answer only if its reasoning is also valid, and argued RL does improve reasoning under that measure. "Spurious Rewards" (Shao et al. 2025) found that Qwen2.5-Math improves even with random rewards, so small effects on that model family are suspect.

3.6 Positive versus negative reinforcement

Zhu et al. ("The Surprising Effectiveness of Negative Reinforcement in LLM Reasoning", NeurIPS 2025) trained with only negative samples (NSR) or only positive samples (PSR). They report that NSR preserves entropy and pass@k while PSR collapses them, credit the negative samples (pushing down wrong answers redistributes probability to other candidates), and recommend W-REINFORCE. They used 4B to 8B models and k up to 256.

3.7 Objectives aimed at pass@k

PKPO (Walder and Karkhanis 2025) and advantage-shaping work (Thrampoulidis et al. 2025) give unbiased gradient estimators for pass@k, so a model can be trained directly for diversity.

3.8 The gaps

Gap in the literature	What this project does about it
The *noise* of adaptive answer-collection rules (random group sizes, early stopping) is not characterized; only the average is.	An exact two-moment theorem: average and noise of any such rule from four numbers per question (Section 5).
Nobody measures gradient size separately for right and wrong answers; recent papers assume they are equal.	We measure it: wrong answers carry 2.4 to 3.1 times larger squared gradients (Section 6).
Adaptive-rollout gains are compared at unequal compute, with one seed.	All our comparisons are at matched generated answers, paired per problem, pre-registered, with at least two seeds.
The NSR explanation (negative samples preserve diversity) was never separated from the fact that NSR and PSR differ in noise.	We show their average updates are identical, measure their noise, and compare diversity at *matched accuracy* (Sections 9–12).
"Does RL preserve diversity?" is usually answered at a fixed step, which confuses slower learning with preserved diversity.	We compare runs at equal pass@1: an accuracy-versus-diversity curve.

4 The earlier idea (STOP) and why it failed

Why we did it

Before this project, the plan was STOP: a better way to give credit inside an answer. Segment-level RL splits each answer into blocks and credits every block with the change in estimated success across it. STOP asked *where the blocks should end*. Existing methods pick cut points with local scores (token entropy, value variance, EPIG-Tree's gradient-aware score). We showed with exact small examples that a local score can pick the wrong cut, because a cut changes its neighbors and because token gradients interact, and planned a geometry-aware cut rule.

The theory worked. The update error splits exactly into a part that ignores gradient geometry and an interaction part; the interaction vanishes when token gradients are orthogonal; a cut's benefit depends on the other cuts only through its two neighbors.

What we found

In Qwen2.5-Math-1.5B, gradients at different positions are almost orthogonal (median $|\cos| \leq 0.005$) and the small interactions cancel in aggregate (about $\pm 1\%$). In a pre-registered test over all partitions of a 64-token grid, interactions reversed only 2.2% of orderings, none statistically clear, and never changed the best single cut; the geometry-free score lost at most 1.3% of the available gain. The effect STOP was built to exploit is real in constructed examples but too small in a real model to matter. Written up honestly as "local scores suffice" (TMLR or workshop level).

What we kept: exact per-answer gradient measurement tools, and the habit of *measuring the mechanism before building a method*. Both made the next project possible.

5 What is each generated answer worth? (WORTH)

Why we did it

Generating answers is the main cost of RLVR, and the adaptive-collection literature (Section 3) compares rules by heuristics. If the value of each answer could be computed exactly, rules could be ranked, and the best rule derived instead of guessed.

5.1 The two-moment theorem

Let p be a question's success probability, F_1 and F_0 the average squared score size $\mathbb{E}\|\ell\|^2$ of right and of wrong answers, and $\|\nabla p\|^2$ the size of the question's learning signal. Two simple facts drive everything: a right answer's average score is $\nabla p/p$ and a wrong answer's is $-\nabla p/(1-p)$; and given its reward, an answer is independent of the others.

Theorem (two moments of any reward-adapted rule)

For any rule that decides when to stop sampling and which advantages to give using only the rewards seen so far (including random group sizes, early abandonment, sampling until success, and filtering),

$$\mathbb{E}g = c(p)\,\nabla p, \qquad \mathbb{E}\|g\|^2 = F_1 D_1(p) + F_0 D_0(p) + \|\nabla p\|^2 X(p),$$

where c, D_1, D_0, X and the expected cost depend only on the rule and on p. So the average, the noise and the cost of the update depend on a question only through **four numbers**: p, $\|\nabla p\|^2$, F_0, F_1.

Why it is true, in one line: condition on the whole sequence of rewards; then the stopping time and the advantages are fixed numbers and the answers are independent, so averaging over answers and then over rewards gives both formulas. No optional-stopping argument is needed. **What is new** is the noise part with separate F_0, F_1 and random group sizes; the average part for fixed groups was known (Davis and Recht; ARCUS). All statements are verified by exact enumeration in small models (`theory/exact.py`).

5.2 Consequences: ranking the rules and the optimal rule

Treating questions as having unrelated gradients and choosing the step size optimally, the progress per generated answer is proportional to signal^2/(cost $\times$ second moment). Dynamic programming over the counts of (wrong, right) answers seen so far gives the **optimal rule**: after each block of 4 answers, a lookup table says stop or continue, and which advantages to give.

Rule (efficiency relative to the optimal rule)	Uniform	Mostly hard	U-shaped	Mostly easy
Fixed group of 8 (GRPO, Dr. GRPO, DAPO)	0.61	0.38	0.40	0.38
Fixed group of 16	0.65	0.41	0.43	0.41
Probe 4, abandon if all agree, fill to 16	0.77	0.60	0.67	0.60
Sample until 1 correct, cap 16	0.26	0.09	0.12	0.30

Table 1: Exact efficiency by difficulty mix. DAPO's filtering has exactly the efficiency of the group it filters; sample-until-correct is worst; probe-then-fill is the best rule in use.

5.3 When does a better rule matter? The critical budget

Without assuming unrelated question gradients, one step's progress at B answers per step is $\langle \nabla J, \mu \rangle^2 / (2L(s + nV/B))$, with s the squared size of the mean update and V its noise. The **critical budget** $B^\star = nV/s$ splits two regimes: below it, a rule k times more efficient is worth k times the answers (about $\sqrt{k}$ under Adam at a fixed learning rate); above it, no reweighting helps (Figure 1). This later turned out to be already published for GRPO (Bansal et al.), so it became background.

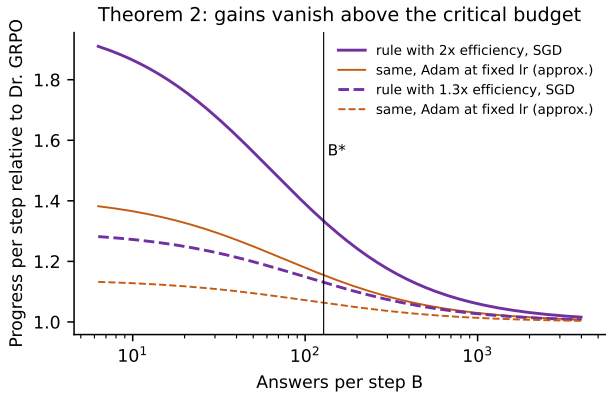


Figure 1: Predicted gain of a better rule against answers per step: gains vanish above the critical budget, and Adam turns a gain of k into about $\sqrt{k}$.

6 Measuring the four numbers

Why we did it

The theory's predictions depend on two unknowns: how F_0 compares with F_1, and how the learning signal $\|\nabla p\|^2$ varies with difficulty. Both had to be measured in a real model before any prediction meant anything.

What we found

Wrong answers carry larger gradients. In Qwen2.5-Math-1.5B (exact per-answer gradients on MATH training problems): in the last-layer block, 40 questions by 16 answers, $F_1/F_0 = 0.36$ (90% intervals do not overlap), while wrong answers are only 9% longer (421 against 386 tokens). Over all 1.5B parameters, measured before every training run: F_1/F_0 between 0.32 and 0.42, so **wrong answers carry 2.4 to 3.1 times larger squared gradients**. Likely reason: wrong answers contain more low-probability tokens, whose scores are large. This contradicts VSR's Assumption B.3.

The learning signal fades at both ends. With 96 answers per question, the estimated $\|\nabla p\|^2$ is 0.73 ± 0.60 on very easy or very hard questions and 15.5 ± 5.4 on medium ones; the best fit is $p^2(1-p)^2$, and a flat shape is rejected (Figure 2). This makes the optimal rule robust: it should spend answers on medium questions.

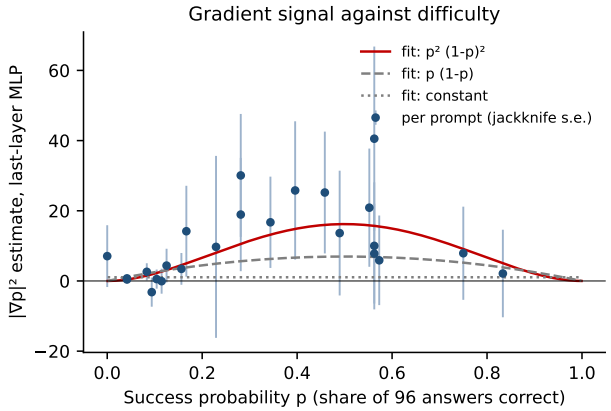


Figure 2: Per-question learning signal against success probability, with three candidate shapes.

7 First tests of the theory

7.1 Pre-registration 1: scale down wrong answers (pilot)

Why we did it

The simplest consequence of $F_0 > F_1$: weight each wrong answer's advantage by $w = F_1/F_0$. A one-line change to Dr. GRPO, predicted to raise efficiency by 1.1 to 1.6 times in 11 of 12 settings.

Design. Qwen2.5-Math-1.5B base, 3,920 numeric MATH training problems (four subjects), 16 questions by 8 answers per step, 150 steps, 512-token answers, AdamW at 10^{-6}; Dr. GRPO against the scaled version; two seeds; primary: greedy accuracy on the 364 numeric MATH-500 problems. **Prediction:** promising if at least +2 points in both seeds.

What we found

No gain: 45.3% against 47.5% (seed 1), 44.0% against 46.7% (seed 2). Verdict: **no evidence**. Lesson: under Adam at a fixed learning rate, a 1.1 to 1.6 times efficiency gain shrinks to 1.06 to 1.25 times, too small to see with two seeds.

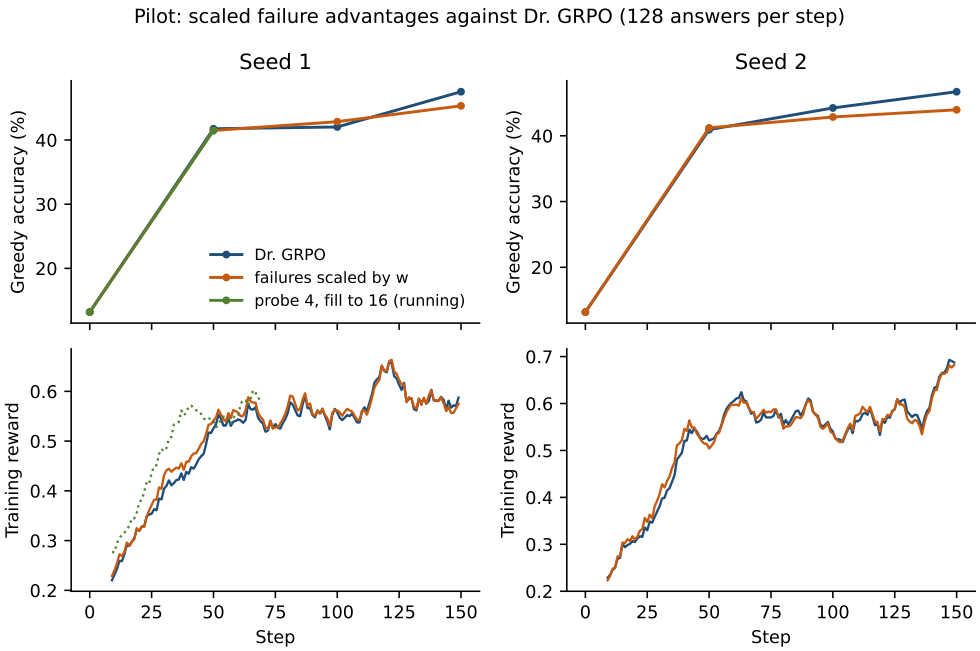


Figure 3: Pilot: scaled failures (orange) track Dr. GRPO (blue).

7.2 Pre-registrations 2 and 3: collection rules at equal cost, and measuring noise inside training

Why we did it

The optimal rule predicts a larger gain (1.7 to 2.3 times Dr. GRPO's efficiency). Testing it fairly needs exactly matched generated answers. And to know *why* a rule helps or not, we needed to measure signal and noise during training, not just accuracy.

Design. The optimal rule (lookup table re-solved every 10 steps from the recent difficulty mix), probe-then-fill, scaled failures, and Dr. GRPO at several budgets (32 to 256 answers per step), all at matched answers (19,200 over 150 steps for the 128-answer arms), two seeds, paired per problem on 3,382 held-out MATH test problems. **New tool:** at every step the questions are split at random into two halves; inner products of the two halves' gradients, and of each half with an independent unbiased reference for the population gradient, give the signal, the noise and the noise scale, without extra sampling. **Prediction:** promising if the optimal rule beats Dr. GRPO by at least 1.5 points in both seeds.

What we found

- Optimal rule: $+1.1 \pm 0.4$ (seed 1) and $+3.3 \pm 0.5$ (seed 2) points, with matched answers (19,172 and 19,200) and about 1% more tokens. Positive in both seeds, but the pre-registered bar (1.5 in both) was **missed** because of seed 1.
- Probe-then-fill: +2.0 and +0.8 (MATH-500). Scaled failures: −0.4 and +1.1. Fewer answers clearly hurts: 32 against 128 answers per step, −4.4 points (seed 1; not every rung of the budget ladder obtained GPUs).
- Training becomes noise-limited as it plateaus: Dr. GRPO's noise scale grows from about 900 answers (steps 0 to 49) to over 10,000 (steps 50 to 149): the shared signal shrinks while the noise does not.
- Gains are much smaller than one-step theory predicts, consistent with Adam's square-root effect.

8 The literature check and the change of direction

Why we did it

Before investing more, we checked carefully (three literature reviews, key claims verified by hand) whether the framing was new.

What we found

- **The critical-budget framing was taken** (Bansal et al., HiLD at ICML 2026), including the finding that within-question noise dominates.
- The common evaluation flaws listed in Section 3.
- **No paper measures gradient norms by correctness**, as far as we found, and none separates the average and the noise of positive and negative reinforcement.

So the main question moved to positive versus negative reinforcement, where our theorem and our measurement give something nobody else has. The optimal-rule result stays as a practical side result.

8.1 The key observation

Give each right answer advantage a and each wrong answer b. The theorem gives

$$\mathbb{E}g = n(a-b)\,\nabla p, \qquad \text{noise} \approx n\big(a^2 p F_1 + b^2(1-p)F_0\big).$$

- PSR $(a = 1, b = 0)$ and NSR $(a = 0, b = -1)$ both have average $n\nabla p$: **exactly the same expected update**. (This is the classical baseline invariance; the new part is the noise and its measurement.)
- In fact NSR = PSR $- \sum_i \nabla \log \pi(y_i)$, and the extra term has average exactly zero: it is pure noise.

- PSR's noise npF_1 sits on easy questions; NSR's $n(1-p)F_0$ on hard ones. With $F_0 \approx 2.5F_1$, NSR is noisier on every question solved less than 71% of the time.
- The whole family $A = r - b$ shares the average; its noise is smallest near $b^\star = pF_1/(pF_1 + (1-p)F_0) \approx 0.15$.

Consequence: to first order, any difference between PSR and NSR (entropy, pass@k) must come from the noise, from an optimizer that reacts to noise (Adam), or from implementation choices outside the theorem.

9 PSR against NSR (Pre-registration 4)

Why we did it

Test the observation directly: if NSR's famous diversity advantage is real in clean on-policy training with equal averages, it must be a noise effect; if it does not appear, it must come from implementation.

Design. Dr. GRPO, PSR, NSR, W-REINFORCE; two seeds; strictly on-policy; in-training noise measurement; an on-policy entropy probe (the first four answers of every question, identical for all rules); pass@k with 64 samples at step 150. **Prediction:** separation if NSR beats PSR by at least 3 points of pass@64, or has at least 20% more entropy, in both seeds.

Rule (seed 1 / seed 2)	**Training acc., steps 25–49**	**pass@1 at 150**	**pass@64 at 150**	**Entropy, 100–150**
Dr. GRPO	50.6 / 48.0	43.3 / 43.2	83.2 / 81.9	0.227 / 0.224
PSR (reward right only)	51.3 / 48.2	42.4 / 42.0	83.2 / 83.8	0.227 / 0.226
NSR (penalize wrong only)	33.6 / 36.4	37.7 / 39.1	83.0 / 82.1	0.251 / 0.251
W-REINFORCE	38.6 / 38.0	38.5 / 39.8	80.5 / 81.3	0.243 / 0.256

Table 2: Untrained model: pass@1 13.8, pass@64 79.9.

What we found

NSR's measured noise is 6.4 times PSR's (theory about 7). NSR learns clearly more slowly, ends with lower pass@1, and reaches the *same* pass@64; its entropy is only 11% higher. The measured noise matches the theorem within 30% for every rule. Verdict: **the separation is not reproduced** in either seed. W-REINFORCE, the NSR paper's recommendation, has the lowest pass@64.

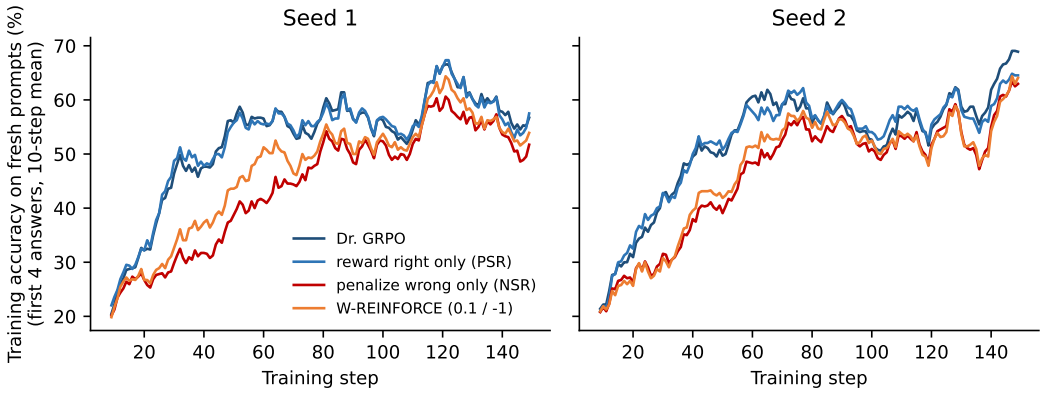


Figure 4: Training accuracy on fresh questions. The noisy rules, dominated by wrong answers, learn about twice as slowly.

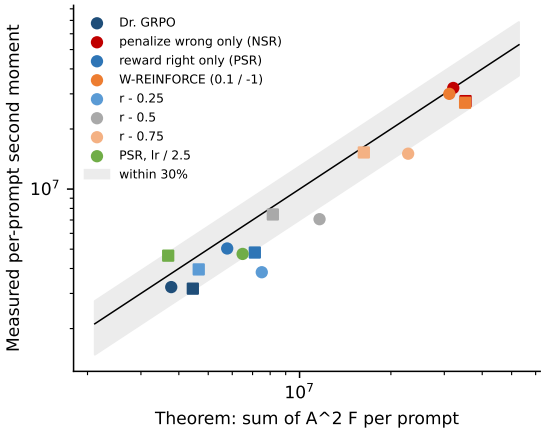


Figure 5: Measured gradient noise against the theorem's prediction from F_0, F_1 and the observed reward counts; shaded: within 30%.

10 Matched accuracy, and the NSR paper's own implementation

10.1 Pre-registration 5: is NSR just a slower PSR?

Why we did it

At a fixed step, a slower run looks more diverse simply because it has learned less. The fair comparison is diversity *at the same accuracy*: for each NSR checkpoint, interpolate PSR's pass@32 at the same pass@1 along PSR's trajectory and take the difference. This gives an **accuracy-versus-diversity curve** per run.

Design. PSR and NSR, seeds 3 and 4, pass@32 from 32 samples on 128 MATH-500 problems at steps 0 to 150. **Prediction:** the matched difference is below 2 points in both seeds.

What we found

NSR is 2.1 points *below* PSR's curve (seed 3) and 0.2 below (seed 4). **No extra diversity at equal accuracy** (Figure 6, left).

10.2 Pre-registration 6: do the NSR paper's implementation choices create the effect?

Why we did it

Zhu et al. used verl, which differs from clean training in three ways the theorem does not cover: each batch of answers is reused for 4 updates with clipping (stale samples), each answer's loss is averaged over its tokens, and PSR and NSR average over only the answers they train on. If the effect lives anywhere, it may live there.

Design. All three choices reproduced at matched answers and updates (with fresh samples, our clipped loss equals the plain loss exactly). **Prediction:** recreated if NSR beats PSR by 3 points of pass@32, or 20% entropy, in both seeds.

What we found

NSR learns even more slowly here (training accuracy 31.0 and 31.4 against 43.0 and 45.4), and at step 150 its pass@32 is 7.0 and 1.6 points *lower* than PSR's; its entropy is 1.11 and 1.18 times PSR's; at matched pass@1 the differences are +0.3 and −1.3. Verdict: **not recreated**. Disclosure found later: per-token averaging makes gradients larger, so the gradient clipping at norm 1 (verl's default) was active on 26 to 28% of NSR's steps and 7 to 8% of PSR's; it never triggered in any other on-policy run.

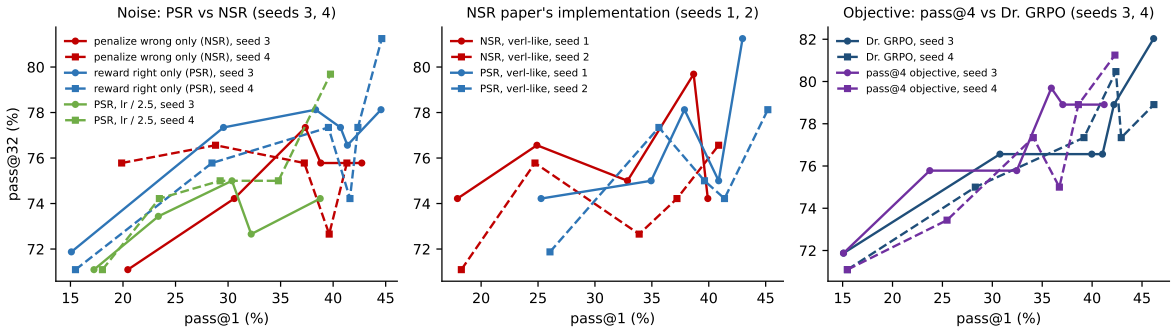


Figure 6: Accuracy (pass@1) against diversity (pass@32) during training, 128 problems. Left: changing the noise (PSR, NSR, PSR at a lower learning rate). Middle: the NSR paper's implementation. Right: changing the objective (pass@4 against Dr. GRPO). No run separates from its reference beyond the evaluation noise (point comparisons, not yet an equivalence test).

11 Same average, graded noise (Pre-registration 7)

Why we did it

The strongest test of "only noise differs" available with one family: the baselines $A = r - b$ for $b = 0, 0.25, 0.5, 0.75, 1$ all have exactly the same average update, but the theory predicts different noise. If learning speed follows the predicted noise, noise is the likely cause. A PSR run with the learning rate divided by 2.5 tests the "slower clock" reading: does NSR look like PSR run more slowly?

	$b = 0.25$	$b = 0$ (PSR)	$b = 0.5$	$b = 0.75$	$b = 1$ (NSR)	PSR, lr /2.5
Predicted noise vs PSR	0.81×	1×	1.25×	2.5×	about 6×	1×
Steps to 45% training accuracy, seed 3	37	44	44	53	56	84
Steps to 45% training accuracy, seed 4	38	33	41	45	56	76
pass@32 vs PSR at equal pass@1, seed 3	+0.0	0	+0.8	−0.8	−2.1	
pass@32 vs PSR at equal pass@1, seed 4	+0.2	0	+2.1	+0.6	−0.2	

What we found

- **Speed follows predicted noise** in both seeds (rank correlation 0.9 to 0.97; pre-registered threshold 0.8): passed.
- The noise formula is within 30% in all seed-4 runs but only 0.51 to 0.73 in seed 3, so 12 of 16 runs overall; likely because gradient sizes are measured once, before training, and shrink as the model sharpens.
- Every run stays within about ±2 points of PSR's curve; "noise adds exploration" is rejected. The strict slower-clock rule asked for every difference below 2 and two were 2.07 and 2.14: **formally inconclusive**. PSR at a 2.5 times lower learning rate learns *more* slowly than NSR, so NSR equals a smaller slowdown.

12 Changing the objective, and grading the reasoning (Pre-registration 8)

Why we did it

If noise only changes speed, perhaps changing *what the update aims at* moves the curve. We aimed the update exactly at pass@4: per-answer weights from the unbiased pass@k estimator with a leave-one-out baseline, verified over all 256 reward patterns to average $2 \cdot 4(1-p)^3 \nabla p$, the pass@4 gradient (such objectives are known, e.g. PKPO; here it is a control). It pushes hard questions and leaves solved ones alone. We also graded the reasoning, not just the final number, because a diversity gain could hide lucky guesses.

Prediction: the curve moves if pass@4 training is at least 2 points of pass@32 above Dr. GRPO at equal pass@1 in both seeds.

What we found

+1.6 (seed 3) and +0.2 (seed 4): **curve fixed**. At step 150 pass@4 training simply has lower pass@1 (41 to 42 against 46). With reasoning graded by Claude Haiku 5.5 (three judgments per correct answer, majority vote; cost about $9):

Step 150, seed 3 / seed 4	pass@1	CoT-Pass@1	pass@32	CoT-Pass@32
Dr. GRPO	46.2 / 46.2	43.4 / 43.3	82.0 / 78.9	74.2 / 70.3
pass@4 objective	41.2 / 42.3	38.6 / 39.3	78.9 / 81.2	72.7 / 72.7
Untrained model	15.1	13.5	71.9	66.4

Reasoning-checked diversity is the same (CoT-Pass@32 72.7 against 72.3 on average); about 93% of trained models' correct answers have valid reasoning, up from 90%.

13 What the Qwen experiments say together

The picture after eight pre-registrations

In on-policy RLVR on Qwen2.5-Math-1.5B, every reward-shaping choice we tried (PSR, NSR, five baselines in between, the NSR paper's implementation, and a pass@4 objective) moved the model *along* one accuracy-versus-diversity curve at different speeds; none moved the curve beyond about 2 points. The speed followed the noise that the theorem predicts from four numbers per question. NSR's "preserved diversity" is what any slower run looks like at a fixed step.

Two side results: the theory explains why standard practice is close to right (Dr. GRPO's per-question baseline is 1.25 times less noisy than the best possible constant baseline), and the optimal answer-collection rule moves faster along the curve at equal cost. A planned method, WORTH-k (a noise-minimized pass@k objective), was dropped: it assumed that changing the objective moves the curve, and Pre-registration 8 found it does not at this scale.

14 An independent review, and what we changed

Why we did it

The goal is an ICML paper. Before spending more, we asked an independent model (Codex) to review the project as a demanding ICML reviewer and area chair, and since then it reviews every experiment before launch.

What we found

Verdict: **reject as it stands, 4 out of 10 (about 10 to 20% acceptance), with a credible path to a main-track paper**; about 6 out of 10 (30 to 45%) if a second model family establishes one curve with intervals; 5 out of 10 if the curves robustly differ. Its main points, all accepted:

1. **Novelty is narrower than framed.** Equal averages for PSR and NSR are classical baseline invariance; pass@k objectives exist (PKPO); Dr. GRPO, the coverage framing (Yue et al.) and CoT-Pass@k (Wen et al.) are prior work. Ours: the noise accounting with $F_0 \neq F_1$ and adaptive group sizes, the measured asymmetry, and the matched-accuracy tests.
2. **"No difference detected" was presented as "the same".** One 1.5B model, two seeds per comparison, 128 problems, point estimates; the NSR paper used 4B to 8B models and k up to 256.
3. **Noise predicts speed, but cause is not shown.** The baseline family changes the noise and the push on wrong answers together.
4. Four numbers give the *size* of the noise, not its geometry or what Adam does with it.

What we changed (no GPU needed): the report, the collaborator brief and the story now credit baseline invariance and PKPO, drop every "nobody measured this" claim, describe the Qwen result as absence of detected separation, state the scale gap to the NSR paper, and disclose the clipping in Pre-registration 6. **What we launched:** the two experiments below, each reviewed by Codex several times (Pre-registration 9 four times, Pre-registration 10 three times) and changed after every review.

15 The two experiments running now

15.1 Pre-registration 9: does the picture hold for another model family?

Why we did it

Qwen2.5-Math reacts even to random rewards, so a reviewer will discount small effects on it. The claim must hold on a different family, for longer training, with a larger evaluation and proper equivalence intervals.

Design. OLMo-2-1B-SFT (AllenAI, 1.48B parameters, never RL-trained; the launcher checks the architecture), all seven MATH training subjects (5,685 numeric problems, so 300 steps never repeat a question), **300 steps**, answers up to **1,024 tokens** (a GPU smoke test found 37 to 62% of OLMo's answers cut off at 512, which would let format learning pass for accuracy). Five rules, two seeds: Dr. GRPO, PSR, NSR, the pass@4 objective, and the optimal rule at 128 answers per step. Checkpoints every 50 steps; separate evaluation jobs then sample 32 answers per problem on a fixed, subject-stratified set of 256 held-out MATH problems for the base model and every checkpoint, and greedy accuracy on 3,382 problems at step 300.

Decision rules (pre-registered):

1. Asymmetry: F_0/F_1's 95% interval (over questions) above 1.2 in both seeds.

2. Noise and speed: NSR's measured noise above PSR's (interval above 1), and NSR slower to reach the midpoint of PSR's improvement, in both seeds.
3. **Primary, average matched-accuracy equivalence:** pass@16 at matched pass@1 (NSR against PSR, pass@4 against Dr. GRPO), pooled over seeds, 2,000 bootstrap draws over the 256 problems; equivalent if the 90% interval lies within ±2 points, different if the 95% interval excludes 0 and the estimate is at least 2 points; otherwise inconclusive; incomplete evaluation and insufficient overlap are separate verdicts. A simultaneous band over all checkpoints is reported so crossings cannot hide in an average.
4. Optimal rule minus Dr. GRPO at equal answers (budget checked within 2%).

Status. All 10 runs are training (about 90 to 115 seconds per step; done in about 6 to 8 hours), followed by about 3 hours of evaluation jobs. At step 50, greedy MATH-500 accuracy had moved from 12.9% to 15 to 19%.

15.2 Pre-registration 10: is the noise the cause?

Why we did it

This answers the review's causal objection. In the baseline family, more noise and a harder push on wrong answers always come together. We need an arm with NSR's noise but without NSR's push.

The intervention, PSR+N. Give every answer advantage $r_i + c_t s_i$, where s_i is a random sign (±1 with probability one half). Over the signs, the added term averages to zero, so the expected update is exactly PSR's and NSR's. The size c_t is set at each step so that the theorem's predicted noise equals NSR's on that step's answers. So PSR+N has NSR's noise size but pushes wrong answers neither up nor down systematically.

- If PSR+N learns as slowly as NSR: trace-matched noise reproduces NSR's lag, so noise size is enough to produce it (noise geometry and Adam's reaction remain possible routes).
- If PSR+N learns clearly faster: noise size alone does not account for the lag; something about NSR's specific update matters (without telling us what).

Design. Qwen2.5-Math-1.5B, the earlier protocol, fresh seeds 5, 6, 7; per seed four arms run in parallel on one 4-GPU machine: PSR, NSR, PSR+N, and NSR at twice the learning rate; plus PSR at twice the learning rate for seeds 5 and 6 (a fair comparison at the higher rate). All arms of a seed see the same questions at every step, so comparisons are paired. Extra measurements: an NSR "shadow" computed on PSR+N's own answers at the same moment (to check the noise really matches), F_0/F_1 re-measured at steps 50, 100, 150 on the same questions, and two Adam statistics (the gradient scaled by Adam's denominator, and how well each actual Adam step aligns with the true improvement direction).

Decision rules (pre-registered). Speed ϕ places a run between NSR (0) and PSR (1) using training accuracy over steps 25 to 49, pooled over the three seeds, with a paired block bootstrap (a power check on old runs gave interval half-widths of about 0.1). Manipulation check: PSR+N's measured noise within a factor 1.3 of the shadow NSR's. Primary: ϕ(PSR+N)'s 90% interval inside $\pm 1/3$ means "reproduces NSR's lag" (the margin is where the $b = 0.75$ run sat); 95% interval above 1/3 means "clearly faster". Also: the first formal equivalence test for NSR against PSR on Qwen

(three fresh seeds as the primary test; adding seeds 3 and 4 as a disclosed secondary), and whether doubling the learning rate recovers NSR's lag.

Status. A 6-step smoke test on the real GPU passed (peak memory 35 of 48 GB, all statistics logged, checkpoint saved). Seeds 5, 6 and 7 and PSR at twice the rate for seed 6 are training; PSR at twice the rate for seed 5 is waiting for a GPU. About 8 hours.

16 Where we stand, and the path to a paper

16.1 The claim we would submit

Claim (if both running experiments pass)

In on-policy RLVR, positive and negative reinforcement share their expected update. NSR's extra term is zero-mean noise concentrated on wrong answers, which carry 2.5 to 3 times larger gradients. The size of that noise, predicted from four numbers per question, sets how fast a run moves along an accuracy-versus-diversity curve that reward shaping does not move. The optimal answer-collection rule is the practical consequence: it moves faster along the curve at equal cost.

16.2 Evidence status

Piece	Status	Evidence
Two-moment accounting	Established	Exact proof, exact enumeration
Wrong answers carry larger gradients	Established on Qwen	Every run; OLMo smoke test 2.3; P9 tests it formally
Noise formula matches real training	Mostly	12 of 16 runs within 30%; P10 re-measures F
Noise predicts speed	Supported, correlational	Dose-response, two seeds
Noise *causes* NSR's lag	Open	P10 (running)
Rules share one curve	Not detected otherwise; equivalence untested	Point comparisons, 128 problems; P9 and P10 test with intervals
Holds beyond Qwen	Open	P9 (running)
Optimal rule faster at equal cost	Partial	+1.1, +3.3; threshold missed; P9 repeats it

16.3 What each outcome means

OLMo (P9)	Noise-only test (P10)	Paper
Equivalence holds	PSR+N as slow as NSR	The full claim; ICML submission (review estimate about 6 of 10)
Equivalence holds	PSR+N faster than NSR	"One curve", and NSR's slowness is more than noise size: a sharper, still interesting paper
Curves differ	either	A boundary result: where reward shaping does move the curve, explained by the measured noise or not; about 5 of 10
Inconclusive	either	TMLR as a carefully scoped diagnostic paper

16.4 Next steps

1. When P9's training ends: launch its evaluation jobs (already reviewed), run the pre-registered analysis, have Codex review the results.
2. When P10 ends: run its analysis, Codex review.
3. Write the ICML draft around the single claim (can start now, with slots for the new results).
4. If both hold: larger evaluation at a few overlapping accuracy levels from saved checkpoints, longer PSR and NSR runs, one larger model if it fits on one GPU.

16.5 Cost so far

About 118 GPU-instance hours (roughly $480) before the afternoon of October 17, about 30 runs since (a few hundred dollars), $9 of LLM judging; Pre-registration 9 about $350 and Pre-registration 10 about $280 when finished. All jobs run on SageMaker (single L40S GPUs, or one 4-GPU machine per seed in P10).

A Experiment tracker

#	Question	Why	Design and prediction	Result and verdict
S	STOP: do token-gradient interactions change the best cut?	Basis of a geometry-aware credit method	Exact per-token gradients, all partitions of a 64-token grid	Interactions tiny; best cut never changed. **Idea dropped**
M	Do right and wrong answers have different gradient sizes?	The theory's predictions hinge on it	Exact per-answer gradients, last layer and full model	Wrong answers 2.4 to 3.1× larger squared gradients. **Established**

#	Question	Why	Design and prediction	Result and verdict
M	How does the learning signal vary with difficulty?	Shapes the optimal rule	96 answers per question	Fades at both ends ($p^2(1-p)^2$). **Established**
1	Does scaling down wrong answers help?	Simplest consequence of the asymmetry	Dr. GRPO vs scaled, 2 seeds; ≥ 2 points in both	-2.2, -2.7. **No evidence**
2	Optimal rule and probe-then-fill at equal answers	Theory predicts 1.7–2.3× efficiency	Matched answers; ≥ 2 points	Probe +2.0, +0.8; optimal moved to 3
3	When does collection matter? Measured noise	Explain gains or their absence	Budget ladder, optimal rule, in-training noise; ≥ 1.5 in both	Optimal $+1.1 \pm 0.4$, $+3.3 \pm 0.5$ (**threshold missed**); fewer answers -4.4; noise scale 900 $\rightarrow$ 10,000+
4	PSR vs NSR vs Dr. GRPO vs W-REINFORCE	Same average; is NSR's effect a noise effect?	2 seeds, on-policy; ≥ 3 pass@64 or $\geq 20\%$ entropy	pass@64 equal; NSR 6.4× noisier, slower. **Not reproduced**
5	Same diversity at equal accuracy?	Fixed-step comparisons confuse speed and diversity	Matched pass@1, seeds 3, 4; $\lvert\text{diff}\rvert < 2$	-2.1, -0.2. **No extra diversity**
6	Do the NSR paper's implementation choices create it?	Reuse, clipping, token averaging are outside the theorem	verl-like, 2 seeds; ≥ 3 pass@32 or $\geq 20\%$ entropy	NSR -7.0, -1.6 at step 150. **Not recreated** (clipping active, disclosed)
7	Does speed follow predicted noise?	Same average, graded noise	5 baselines + PSR at lr/2.5, 2 seeds; rank corr. ≥ 0.8	0.9–0.97, **passed**; slower clock **inconclusive** (2.07, 2.14)
8	Does changing the objective move the curve?	Test the other half of the principle	pass@4 vs Dr. GRPO, 2 seeds; ≥ 2	+1.6, +0.2, CoT-Pass@32 equal. **Curve fixed**
R	Is it an ICML paper?	Independent check before spending more	Codex as ICML reviewer	4/10; corrections made; P9, P10 designed
9	Does it hold on OLMo, longer, with intervals?	Generality; Qwen is suspect for small effects	5 rules, 2 seeds, 300 steps, 256 problems, equivalence ± 2	**Running**
10	Is the noise the cause?	Causal test asked by the review	PSR+N, NSR×2, PSR×2, 3 fresh seeds; ϕ within $\pm 1/3$	**Running**

B Glossary

Advantage The weight that pushes a sampled answer up (positive) or down (negative).

Baseline A number subtracted from rewards; a constant one leaves the average update unchanged and changes the noise.

Calibration Measuring F_0 and F_1 on held-out questions before training.

CoT-Pass@k pass@k counting only answers whose reasoning is judged valid.

Critical budget / noise scale The number of answers per step beyond which noise stops limiting progress.

Dr. GRPO Group-based RL with the group mean as baseline and no extra normalizations.

F_0, F_1 Average squared gradient size of a wrong and of a right answer.

Matched accuracy Comparing diversity of two runs at the same pass@1, by interpolating along one run's trajectory.

NSR, PSR Penalize only wrong answers; reward only right answers.

On-policy Answers always come from the current model, one update per batch.

pass@k Share of questions with at least one right answer among k samples.

Pre-registration A dated file fixing design, prediction and decision rule before the run.

Score $\nabla \log \pi(y)$: the direction that makes answer y more likely.

Split-half diagnostics Splitting each step's questions into halves to measure the update's signal and noise in training.

C How we work, and engineering problems fixed

Method. Every experiment: pre-registered; compared at matched generated answers; paired per problem; at least two seeds; an independent review (Codex) before launch since October 18; results reported whether they support the idea or not (most early predictions failed, and that shaped the project).

Engineering issues fixed along the way. GPU out-of-memory during evaluation (gradients kept alive; fixed by freeing them after each update); crashes on long held-out prompts (generation batch now capped by prompt length; the final check of Pre-registration 4 was lost); Hugging Face rate limits from shared AWS addresses (models now loaded from our S3 copy); a grader crash on a sampled `\boxed{1/0}` (fixed and tested); a job wrapper that reported success when a run failed (fixed); GPU capacity shortages (each job launched in several regions at once, keeping the first to start). All code, pre-registrations, results and figures: `github.com/mayman150/rlvr-adaptive`.